

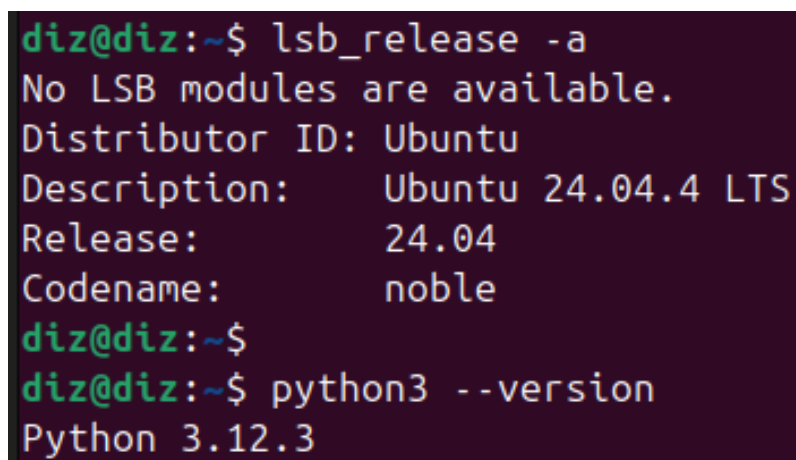
Lab Sheet 1 — Environment Setup

Linux, Python, Git, Mininet & Mininet-WiFi

Course	977-337 Network Programming and Simulation
Student	Hadis Rodpradit
Student ID	6630613039
Program	Digital Engineering (International), College of Computing, PSU Phuket
Environment	Ubuntu 24.04.4 LTS (noble) · Python 3.12.3

This report documents completion of the six lab tasks for Lab Sheet 1, followed by the reflection answers. Each task is evidenced by a terminal / browser screenshot.

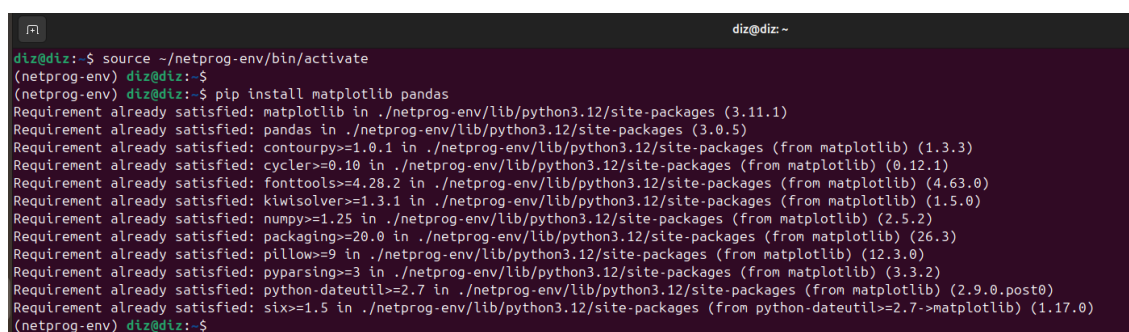
Task 1 (Part A) — OS & Python version



```
diz@diz:~$ lsb_release -a
No LSB modules are available.
Distributor ID: Ubuntu
Description:    Ubuntu 24.04.4 LTS
Release:        24.04
Codename:       noble
diz@diz:~$
diz@diz:~$ python3 --version
Python 3.12.3
```

Figure 1 — `lsb_release -a` confirms Ubuntu 24.04.4 LTS (noble); `python3 --version` reports Python 3.12.3.

Task 2 (Part B) — Virtual environment & packages



```
diz@diz:~$ source ~/.netprog-env/bin/activate
(netprog-env) diz@diz:~$ pip install matplotlib pandas
Requirement already satisfied: matplotlib in ./netprog-env/lib/python3.12/site-packages (3.11.1)
Requirement already satisfied: pandas in ./netprog-env/lib/python3.12/site-packages (3.0.5)
Requirement already satisfied: contourpy>=1.0.1 in ./netprog-env/lib/python3.12/site-packages (from matplotlib) (1.3.3)
Requirement already satisfied: cycler>=0.10 in ./netprog-env/lib/python3.12/site-packages (from matplotlib) (0.12.1)
Requirement already satisfied: fonttools>=4.28.2 in ./netprog-env/lib/python3.12/site-packages (from matplotlib) (4.63.0)
Requirement already satisfied: kiwisolver>=1.3.1 in ./netprog-env/lib/python3.12/site-packages (from matplotlib) (1.5.0)
Requirement already satisfied: numpy>=1.25 in ./netprog-env/lib/python3.12/site-packages (from matplotlib) (2.5.2)
Requirement already satisfied: packaging>=20.0 in ./netprog-env/lib/python3.12/site-packages (from matplotlib) (26.3)
Requirement already satisfied: pillow>=9 in ./netprog-env/lib/python3.12/site-packages (from matplotlib) (12.3.0)
Requirement already satisfied: pyparsing>=3 in ./netprog-env/lib/python3.12/site-packages (from matplotlib) (3.3.2)
Requirement already satisfied: python-dateutil>=2.7 in ./netprog-env/lib/python3.12/site-packages (from matplotlib) (2.9.0.post0)
Requirement already satisfied: six>=1.5 in ./netprog-env/lib/python3.12/site-packages (from python-dateutil>=2.7->matplotlib) (1.17.0)
(netprog-env) diz@diz:~$
```

Figure 2 — With `netprog-env` active, `matplotlib` and `pandas` resolve to `./netprog-env/lib/python3.12/site-packages`, confirming they are installed inside the virtual environment.

Task 3 (Part C) — GitHub repository & initial commit

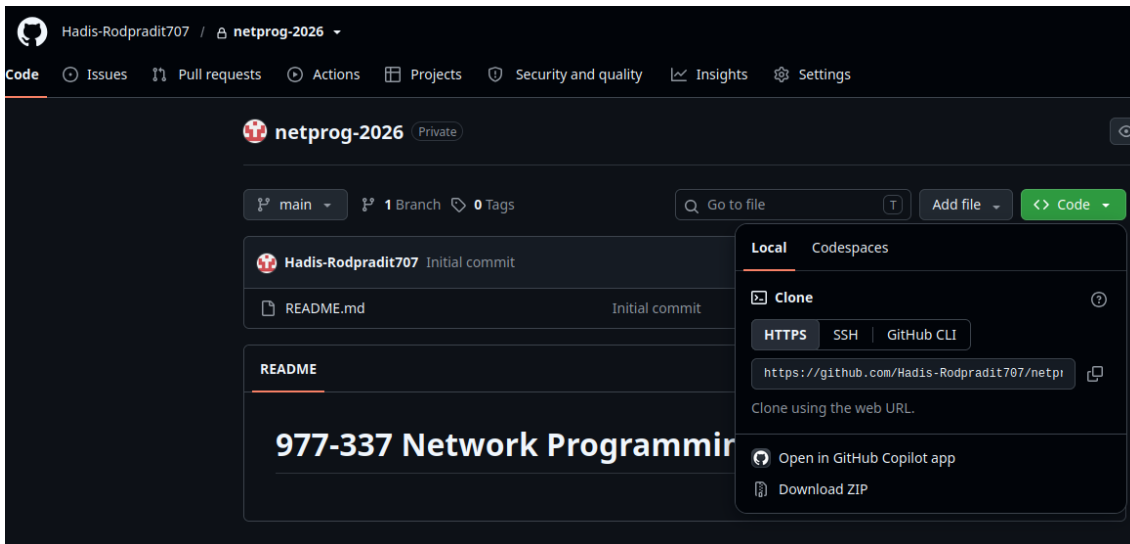


Figure 3 — Repository Hadis-Rodpradit707/netprog-2026 with README.md pushed under the Initial commit on main.

Task 4 (Part D) — Mininet: sudo mn --test pingall

```
diz@diz:~$ sudo mn --test pingall
*** Creating network
*** Adding controller
*** Adding hosts:
h1 h2
*** Adding switches:
s1
*** Adding links:
(h1, s1) (h2, s1)
*** Configuring hosts
h1 h2
*** Starting controller
c0
*** Starting 1 switches
s1 ...
*** Waiting for switches to connect
s1
*** Ping: testing ping reachability
h1 -> h2
h2 -> h1
*** Results: 0% dropped (2/2 received)
*** Stopping 1 controllers
c0
*** Stopping 2 links
..
*** Stopping 1 switches
s1
*** Stopping 2 hosts
h1 h2
*** Done
completed in 5.397 seconds
```

Figure 4 — A 2-host, 1-switch topology is auto-created; pingall reports 0% dropped (2/2 received).

Task 5 (Part E) — Mininet-WiFi sample topology

```
diz@diz:~/mininet-wifi$ sudo python3 examples/simplewifitopology.py
*** Creating nodes
*** Configuring nodes
*** Associating Stations
*** Starting network
*** Running CLI
*** Starting CLI:
mininet-wifi> pingall
*** Ping: testing ping reachability
sta1 -> *** sta1 : ('ping -c1 10.0.0.2',)
PING 10.0.0.2 (10.0.0.2) 56(84) bytes of data.
64 bytes from 10.0.0.2: icmp_seq=1 ttl=64 time=0.737 ms

--- 10.0.0.2 ping statistics ---
1 packets transmitted, 1 received, 0% packet loss, time 0ms
rtt min/avg/max/mdev = 0.737/0.737/0.737/0.000 ms
sta2
sta2 -> *** sta2 : ('ping -c1 10.0.0.1',)
PING 10.0.0.1 (10.0.0.1) 56(84) bytes of data.
64 bytes from 10.0.0.1: icmp_seq=1 ttl=64 time=0.060 ms

--- 10.0.0.1 ping statistics ---
1 packets transmitted, 1 received, 0% packet loss, time 0ms
rtt min/avg/max/mdev = 0.060/0.060/0.060/0.000 ms
sta1
*** Results: 0% dropped (2/2 received)
```

Figure 5 — `simplewifitopology.py` running; `pingall` between wireless stations reports 0% dropped (2/2 received).

Task 6 (Part F) — tcpdump: real packets on sta1-wlan0

```
mininet-wifi> sta1 tcpdump -i sta1-wlan0 -c 10 -v &
mininet-wifi> sta1 ping -c 5 sta2
tcpdump: listening on sta1-wlan0, link-type EN10MB (Ethernet), snapshot length 262144 bytes
PING 10.0.0.2 (10.0.0.2) 56(84) bytes of data:
64 bytes from 10.0.0.2: icmp_seq=1 ttl=64 time=0.545 ms
64 bytes from 10.0.0.2: icmp_seq=2 ttl=64 time=0.335 ms
64 bytes from 10.0.0.2: icmp_seq=3 ttl=64 time=0.221 ms
64 bytes from 10.0.0.2: icmp_seq=4 ttl=64 time=0.161 ms
14:46:04.566055 ARP, Ethernet (len 6), IPv4 (len 4), Request who-has 10.0.0.2 tell 10.0.0.1, length 28
14:46:04.566419 ARP, Ethernet (len 6), IPv4 (len 4), Reply 10.0.0.2 is-at 02:00:00:00:01:00 (oui Unknown), length 28
14:46:04.566433 IP (tos 0x0, ttl 64, id 7109, offset 0, flags [DF], proto ICMP (1), length 84)
    10.0.0.1 > 10.0.0.2: ICMP echo request, id 52131, seq 1, length 64
14:46:04.566577 IP (tos 0x0, ttl 64, id 49084, offset 0, flags [none], proto ICMP (1), length 84)
    10.0.0.2 > 10.0.0.1: ICMP echo reply, id 52131, seq 1, length 64
14:46:05.568090 IP (tos 0x0, ttl 64, id 7928, offset 0, flags [DF], proto ICMP (1), length 84)
    10.0.0.1 > 10.0.0.2: ICMP echo request, id 52131, seq 2, length 64
14:46:05.568329 IP (tos 0x0, ttl 64, id 49778, offset 0, flags [none], proto ICMP (1), length 84)
    10.0.0.2 > 10.0.0.1: ICMP echo reply, id 52131, seq 2, length 64
14:46:06.626145 IP (tos 0x0, ttl 64, id 8978, offset 0, flags [DF], proto ICMP (1), length 84)
    10.0.0.1 > 10.0.0.2: ICMP echo request, id 52131, seq 3, length 64
14:46:06.626333 IP (tos 0x0, ttl 64, id 50835, offset 0, flags [none], proto ICMP (1), length 84)
    10.0.0.2 > 10.0.0.1: ICMP echo reply, id 52131, seq 3, length 64
14:46:07.649062 IP (tos 0x0, ttl 64, id 9231, offset 0, flags [DF], proto ICMP (1), length 84)
    10.0.0.1 > 10.0.0.2: ICMP echo request, id 52131, seq 4, length 64
14:46:07.649198 IP (tos 0x0, ttl 64, id 50911, offset 0, flags [none], proto ICMP (1), length 84)
    10.0.0.2 > 10.0.0.1: ICMP echo reply, id 52131, seq 4, length 64
10 packets captured
10 packets received by filter
0 packets dropped by kernel
64 bytes from 10.0.0.2: icmp_seq=5 ttl=64 time=0.222 ms

--- 10.0.0.2 ping statistics ---
5 packets transmitted, 5 received, 0% packet loss, time 4107ms
rtt min/avg/max/mdev = 0.161/0.296/0.545/0.136 ms
```

Figure 6 — Capturing on sta1-wlan0 shows real ARP and ICMP echo request/reply packets (10 captured, 0 dropped by kernel).

Explanation. The tcpdump capture shows real ARP and ICMP echo request/reply frames—with actual MAC addresses, IP addresses, TTL and sequence numbers—being transmitted on the sta1-wlan0 interface. These are genuine packets moving through the Linux network stack via the mac80211_hwsim driver, not values computed by a mathematical model. A pure simulator would produce no capturable traffic on any real OS interface, so the ability to sniff these frames confirms Mininet-WiFi is emulating (running real networking code), not merely simulating, the wireless link.

Reflection Questions

1. Difference between network simulation and network emulation. Which one is Mininet-WiFi, and how did Part F demonstrate this?

A pure simulator models network behaviour mathematically and only outputs computed numbers—no real packets ever exist. An emulator runs the real operating-system networking code (real sockets, the real TCP/IP stack, software switches), so the same code behaves as it would on real hardware. Mininet-WiFi is an emulator: it uses the mac80211_hwsim module to reproduce 802.11 radio behaviour while still moving real packets over a real data path. Part F demonstrated this because tcpdump on sta1-wlan0 captured actual ARP and ICMP frames with real MAC/IP headers—traffic that could not be captured on any interface in a purely mathematical simulation.

2. Why use a Python virtual environment (netprog-env) instead of installing packages system-wide with pip?

A virtual environment isolates the course's packages and versions from the system Python, so installing or upgrading a library cannot break other projects or the system tools that Ubuntu builds on top of its own Python packages. It keeps the setup reproducible and consistent across machines, avoids needing sudo to install packages, and can be deleted and rebuilt cleanly if something goes wrong—without touching the base OS.

3. What was the most difficult part of the installation, and how did you resolve it?

The hardest part was launching the Mininet-WiFi sample topology on Ubuntu 24.04: the topology failed with a “Please shut down the controller” error because a controller was already bound to TCP port 6653. This was most likely left over from a previous project of mine, which runs its own SDN controller on that same port and had not been shut down cleanly. I resolved it by clearing the leftover network state with `sudo mn -c` before relaunching, which freed port 6653. I also found that the example file was named `simplewifitopology.py` rather than the `simplewifi.py` listed in the sheet, and that `sta1-wlan0` lives inside the station’s network namespace—so I captured packets with “`sta1 tcpdump -i sta1-wlan0`” from the mininet-wifi CLI instead of the host shell.